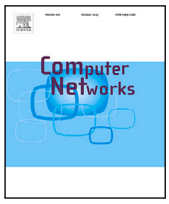
Computer Networks 290 (2026) 112763



Contents lists available at ScienceDirect

Computer Networks

journal homepage: www.elsevier.com/locate/comnet



SaGNN: Encrypted traffic classification with semantic-aware heterogeneous graph neural network

Junjie Luo [a,b], Min Liu [a,b,*], Sheng Sun [a], Yali Chen [a], Qixun Tang [a,b], Yuwei Wang [a]

[a] Institute of Computing Technology, Chinese Academy of Sciences, Beijing, 100190, China
[b] University of Chinese Academy of Sciences, Beijing, 100049, China

ARTICLE INFO

Keywords:
Encrypted traffic classification
Semantic discrimination
Heterogeneous graph
Triple embedding
Graph neural network

ABSTRACT

Encrypted traffic classification is a crucial task in modern cyber security and management. However, the existing methods face two main limitations: (1) At the flow level, they neglect the change in interaction pattern of encrypted traffic, missing the opportunity to explore the semantic differences between distinct communication stages. (2) At the packet level, they lack fine-grained distinction in terms of function and manifestation, leading to semantic conflicts. In this paper, we propose an encrypted traffic classification method with semantic-aware heterogeneous graph neural network (SaGNN). First, we define Heterogeneous Traffic Interaction Graph (H-TIG), which uses different types of nodes and edges to represent packets in distinct communication stages and complex interaction relationships among packets, enabling more accurate and comprehensive representation for encrypted flow. Then, we design Triple Embedding to independently encode byte regions with different semantics within packets, thereby avoiding semantic conflicts at the packet level and enriching node attributes of the H-TIG. Furthermore, we develop a heterogeneous graph neural network, called STGAT, to deeply learn the interaction relationships and node attributes of the H-TIG and obtain an integrated flow-level representation vector, which is subsequently used for classification. Finally, a series of experiments on three well-known datasets demonstrate that SaGNN significantly outperforms representative approaches, achieving an average F1-score improvement of 2.27%.

1. Introduction

With the growing concern over cyber security, traffic encryption technologies have been increasingly and widely deployed. Encryption mechanisms such as the SSL/TLS protocol [1,2] protect user privacy by encrypting packet payload, thereby preventing data leakage during transmission. Nevertheless, the encryption of communication content makes traffic classification more difficult, posing significant challenges to network efficiency optimization and security supervision. On the one hand, Internet service providers (ISPs) and network operators need to identify and classify encrypted traffic to improve network operational efficiency and deliver customized services, thereby enhancing quality of service (QoS). On the other hand, malware and cyber attacks often exploit encrypted traffic to conceal their activities, which substantially increases the difficulty of malicious traffic detection and security supervision. According to existing studies [3], more than 80% of malware propagates via SSL/TLS-based communications. Therefore, effective encrypted traffic classification methods are essential for safeguarding network security.

Due to traffic encryption, plaintext-based classification methods such as deep packet inspection (DPI) [4] are no longer applicable. Attempting to decrypt encrypted traffic before classification would introduce prohibitive computational costs and violate the original intent of user privacy protection. In fact, encrypted traffic can be classified directly. Prior research has demonstrated that encryption algorithms do not achieve ideal randomness, and different types of encrypted traffic exhibit distinct network behaviors, such as variations in network load and latency [5], thus exhibiting distinguishable features.

Methods based on machine learning and deep learning can extract and learn distinguishable features to achieve encrypted traffic classification. Machine learning based encrypted traffic classification methods typically analyze behavioral or statistical features of traffic for classification [6–11], with their accuracy heavily reliant on manually extracted features. In contrast, deep learning based methods leverage inherent automatic feature extraction capabilities to enable end-to-end encrypted traffic classification. These methods model traffic as images or sequences and utilize neural network architectures like Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), or

* Corresponding author at: Institute of Computing Technology, Chinese Academy of Sciences, Beijing, 100190, China.
E-mail addresses: luojunjie21s@ict.ac.cn (J. Luo), liumin@ict.ac.cn (M. Liu), sunsheng@ict.ac.cn (S. Sun), chenyali@ict.ac.cn (Y. Chen), tangqixun23s@ict.ac.cn (Q. Tang), ywwang@ict.ac.cn (Y. Wang).

https://doi.org/10.1016/j.comnet.2026.112763
Received 27 March 2026; Received in revised form 16 August 2026; Accepted 6 September 2026
Available online 20 September 2026
1389-1286/© 2026 Published by Elsevier B.V.

Transformers to extract spatial and temporal features [5,12–15]. Recent studies further improve this paradigm by introducing pre-training and hierarchical representation learning [16–18]. For example, TrafficFormer learns efficient pre-trained representations for traffic data [19], while BTRFormer employs a masked autoencoder with block-based traffic representation to capture hierarchical encrypted traffic features [20]. However, network traffic is not simply an image or a sequence but has complex interaction relationships. The aforementioned deep learning methods overlook these interaction information within the traffic, resulting in an incomplete representation of traffic characteristics. Recently, graph neural networks (GNNs) have been introduced into encrypted traffic classification. Existing GNN-based studies construct traffic graphs at different granularities, including packet-level or byte-level graphs for fine-grained semantic modeling [21,22], flow-level graphs for packet interaction representation [23,24], and cross-flow graph modeling for correlation mining among different flows [25]. Graphs possess strong structural representation capabilities, enabling traffic interaction relationships to be modeled as edges between nodes, thereby overcoming the constraints of Euclidean space and preserving richer original traffic characteristics [26]. Through further learning and aggregation by GNNs, more comprehensive traffic representations encompassing both spatiotemporal and interaction information can be obtained.

Nevertheless, despite these recent advances, existing methods still insufficiently exploit the richer semantics introduced by encryption protocols and face two main limitations:

(1) Neglecting the change in interaction pattern at the flow level: When two parties communicate without encryption, data are transmitted directly (for connection-oriented network protocols such as TCP, a connection must first be established). However, when encrypted communication is employed, due to the integration of encryption protocols, the communication process can be divided into the Handshake Stage and the Transmission Stage. The former negotiates the encryption and compression algorithms and other necessary information used in the encrypted communication, while the latter actually transmits data based on the negotiated configuration. The data packets in these two stages have entirely different content and functions, thereby carrying distinct semantic meanings.

(2) Lack of fine-grained semantic distinction at the packet level: Functionally, the packet header, which includes the link-layer, network-layer, and transport-layer headers, serves to fulfill the fundamental functions of network communication. Packet payload is responsible for carrying data: during the Handshake Stage, it conveys encryption configuration information, while during the Transmission Stage, it carries user information. The packet header, the payload in the Handshake Stage, and the payload in the Transmission Stage each fulfill different functions and thus carry distinct semantics. In terms of manifestation, every packet header is transmitted in plaintext without encryption. For packet payload, the payload in the Handshake Stage is largely unencrypted, whereas the payload in the Transmission Stage is encrypted and appears as ciphertext. Consequently, identical byte appearing in these three distinct regions is likely to possess different semantic meanings.

Motivated by these observations, we propose a method called Semantic-aware Heterogeneous Graph Neural Network (SaGNN) for encrypted traffic classification, which addresses the aforementioned limitations by fully leveraging the richer semantic information introduced by encryption protocols. SaGNN consists of four components: Heterogeneous Traffic Interaction Graph (H-TIG) Construction, Triple Embedding, STGAT for Graph Representation, and Classifier. The H-TIG Construction module transforms encrypted traffic into an H-TIG at the flow level, where different types of nodes and edges represent packets from the Handshake Stage and the Transmission Stage as well as their interactions, thereby highlighting the semantic differences between the two stages. The Triple Embedding module performs fine-grained representation at the packet level to avoid semantic conflicts and enrich the node attributes of the H-TIG. It employs three independent embedding layers to encode the header and payloads of different stages, emphasizing the semantic distinctions of bytes in different regions. In this way, stage-aware semantic modeling in SaGNN is reflected not only in the flow-level H-TIG structure, but also in the packet-level node attributes generated by Triple Embedding. The STGAT for Graph Representation module uses a carefully designed heterogeneous graph neural network called STGAT to hierarchically learn and abstract both the interaction relationships and rich node attributes in the constructed H-TIG, producing a comprehensive flow-level representation vector. This vector is finally fed into the Classifier to obtain the classification results.

The main contributions of this paper are summarized as follows:

- We define the H-TIG to represent an encrypted flow. It preserves the interaction relationships within the flow while distinguishing the semantics between the Handshake Stage and the Transmission Stage.
- We design Triple Embedding, a packet representation method that independently encodes the packet header, handshake payload, and transmission payload, thereby avoiding semantic conflicts and enriching the node attributes of the H-TIG.
- We propose an encrypted traffic classification method named SaGNN, which utilizes a carefully designed heterogeneous graph neural network STGAT to learn the interaction relationships and node attributes of the H-TIG, extracting rich and discriminative semantic features to achieve accurate encrypted traffic classification.
- We conduct extensive experiments on the ISCX VPN-nonVPN dataset and USTC-TFC dataset. The proposed method outperforms representative approaches, achieving an average F1-score improvement of 2.27%, which convincingly demonstrates its effectiveness.

The rest of this paper is organized as follows. Section 2 presents the problem formulation and background knowledge of encrypted traffic. Section 3 reviews related work on encrypted traffic classification. Section 4 details the proposed method. Section 5 provides experimental results and analysis. Finally, Section 7 concludes the paper.

2. Preliminaries

2.1. Problem description

Encrypted traffic classification task is aimed to identify the network traffic generated from different sources and classify it into appropriate categories. The categories refer to different applications, web pages, or services, they may also denote various types of malicious traffic in the context of cyber security [21,27].

In this paper, we focus on encrypted traffic classification task at flow level. A flow is a segment of encrypted traffic, consisting of bidirectional packets that share the same quintuple (source IP, destination IP, source port, destination port, transport protocol). Formally, a flow is defined as a sequence $P = [p_1, p_2, \dots, p_i, \dots, p_n]$, where n is the length of flow and $p_i, i \in [1, n]$ is the ith packet in the flow. p_i is presented as $p_i = [b_i^1, b_i^2, \dots, b_i^j, \dots, b_i^m]$, where m is the length of packet i and $b_i^j, j \in [1, m]$ is the jth byte in the packet. Encrypted traffic classification task can be described as learning an end-to-end model F_{model} that takes a flow P as input and outputs the category c to which the flow belongs, i.e.,

$$c = F_{model}(P), c \in C \tag{1}$$

where C is the set of categories.

2.2. SSL/TLS encrypted traffic

To facilitate the understanding of the richer semantics introduced by encryption protocols, we introduce the fundamentals of encryption

Table 1
Systematic comparison of representative GNN-based encrypted traffic classification methods.

Method	Heterogeneity level	Heterogeneity source	Graph type	Graph granularity
TFE-GNN [21]	Packet	Header/Payload	Homogeneous	Packet
DE-GNN [23]	Packet	Header/Payload	Homogeneous	Flow
MH-Net [22]	Packet	Header/Payload	Heterogeneous	Packet
SaGNN	Flow & Packet	Communication stages & Header/Payload	Heterogeneous	Flow

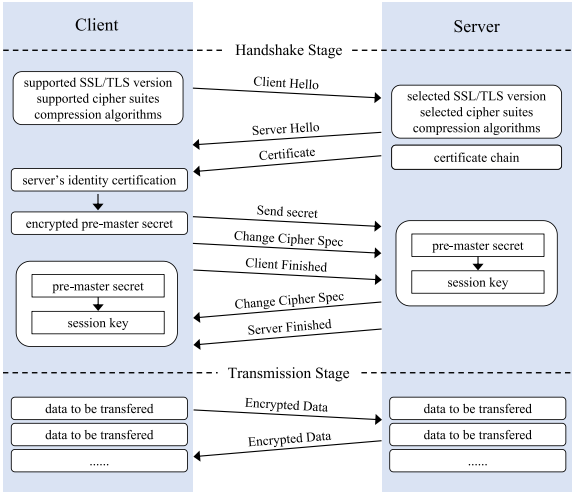


Fig. 1. Two stages of SSL/TLS encrypted traffic.

protocols. Secure Sockets Layer (SSL) protocol [1], or its successor Transport Layer Security (TLS) protocol [2] is the most widely used encryption protocol nowadays. SSL/TLS works between transport layer and application layer, providing data encryption and decryption services for application layer protocols. SSL/TLS is primarily comprised of two sub-protocols, i.e. the Handshake Protocol and the Record Protocol. The Handshake Protocol conducts crucial steps such as identity authentication, selection of encryption algorithms, and key exchange between the communicating parties, while the Record Protocol implements fundamental functions such as encapsulation, encryption, and compression of application layer data using the parameters configured by the Record Protocol. Thus, SSL/TLS encrypted traffic is divided into two stages: the Handshake Stage and the Transmission Stage, as illustrated in Fig. 1.

In the Handshake Stage, the client requests the server to establish an SSL/TLS connection by sending a Client Hello message. The Client Hello message contains the highest version of SSL/TLS version that the client supports, the list of supported cipher suites and compression algorithms. After receiving the Client Hello message, the server informs the client of the selected SSL/TLS version, the selected cipher suites and compression algorithms by sending a Server Hello message. A Certificate message containing a chain of certificates and server's public key is also sent by the server. The client will encrypt its pre-master secret using server's public key once verifying server's identity and send the encrypted secret to the server. After the server decrypts the secret, both the client and the server have the same pre-master secret, and they will generate the same session key based on the pre-master secret and other necessary information using algorithms negotiated. The peers send Change Cipher Spec message and Finished message to announce the completion of the Handshake Stage. In the Transmission Stage, the client and the server transfer application layer data encrypted by the shared session key safely. Notice that most information exchanged in the Handshake Stage is in the form of plaintext, while the data transferred in the Transmission Stage is usually encrypted.

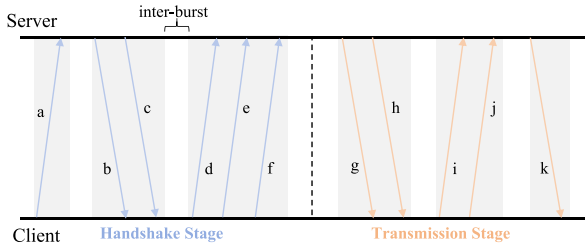
3. Related work

Machine learning based methods classify encrypted traffic using statistical features such as flow duration, packet number, and packet length. These manually selected features serve as input to various classification models. For instance, Saber et al. [6] used Principal Component Analysis (PCA) to select time-based flow feature and performed traffic classification with a Support Vector Machine (SVM), while Zhang et al. [10] used Isolation Forest for classification. Random Forest is also employed in Appscanner [28] and GRAIN [29]. Other studies have designed more structured feature representations, such as Markov probability fingerprints based on message type and packet length [7], multi-scale path signature features derived from packet length [9], and tabular representations informed by Request for Comments (RFCs) [11]. Although these methods can achieve encrypted traffic classification with low overhead, their classification performance is highly correlated with the quality of feature selection. This strong feature dependence restricts their applicability in both cross-protocol and dynamic network scenarios due to feature instability.

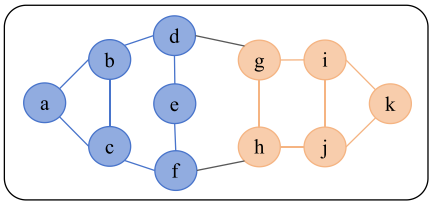
Deep learning based methods typically perform encrypted traffic classification based on byte features or sequence features. Due to their inherent automatic feature extraction capabilities, these methods can achieve end-to-end traffic classification with higher accuracy.

Early studies usually transformed traffic into images or byte sequences and employed CNNs, RNNs, or their hybrid architectures to extract spatial and temporal features [12,13,30–34]. Later works further introduced attention mechanisms and Transformers to enhance the modeling of temporal dependencies and global contextual information in encrypted traffic [5,35–37]. More recently, pre-training and fine-tuning techniques have been explored to learn more general traffic representations from raw traffic data [16–20]. In addition, traffic data augmentation has been adopted to improve robustness under dynamic network conditions [15,38]. Deep learning based methods can automatically extract spatial and temporal features from encrypted traffic for classification. Nevertheless, they struggle to capture fine-grained structural information embedded in packets and the rich interactions among packets within a flow, thus failing to achieve complete representation of the traffic.

Graph neural network based methods model traffic as graphs and leverage graph neural network for traffic representation. Most work constructed graphs and achieved traffic classification at flow level. Shen et al. [39] constructed Traffic Interaction Graph, using packet direction and length as node feature. Han et al. [23] also constructed Traffic Interaction Graph and encoded packet header and payload in a dual manner. Huoh et al. [40] constructed graph solely based on temporal relationship among packets, which failed to fully utilize the structural representation capability of graph. Some studies also treated flow as node when constructing graphs. For instance, Zheng et al. [41] and Lu et al. [42] mapped flow to node of the graph and create edges between flows sharing the same IP. What is more, Zhang et al. [21] used PMI value of bytes to construct graph for packets, thus efficiently obtaining discriminative information contained in raw bytes. As a subsequent work, Zhang et al. [22] constructed multi-view heterogeneous graphs for packets, thus fully utilizing diverse granularities of information and different types of byte correlations. Other methods were designed for robust or privacy consideration. Diao et al. [43] used multi-scale graph convolution networks to improve classification under high packet loss and reordering. Cui et al. [24] constructed Flow Graph based on window relationship and acknowledgment relationship in transport layer, and introduced a Jensen–Shannon divergence (JSD) based feature selection algorithm to select robust node attributes. Pei et al. [44] designed a privacy-enhancement graph representation to



(a) Client–server interaction in an encrypted flow.



(b) Heterogeneous Traffic Interaction Graph (H-TIG).

Fig. 2. Construction of heterogeneous traffic interaction graph. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

protect privacy when conducting network intrusion detection. However, these methods treat all packets within a flow equally and fail to perform fine-grained semantic distinction for packets. Consequently, they cannot fully exploit the richer semantic information introduced by encryption protocols, thereby limiting the potential for encrypted traffic classification.

To more clearly distinguish SaGNN from closely related graph-based methods, Table 1 provides a systematic comparison. Existing methods mostly model heterogeneity at the packet level, mainly through the distinction between packet header and payload. In contrast, SaGNN considers heterogeneity from both the flow level and the packet level. Beyond separating header from payload, it further emphasizes the semantic differences among different transmission stages of encrypted flows. Such a more systematic modeling of heterogeneity and fine-grained semantic separation enables SaGNN to capture richer semantic information in encrypted flows.

4. Methodology

In this section, we detail SaGNN, the proposed semantic-aware heterogeneous graph neural network method for encrypted traffic classification. The input of SaGNN is an encrypted flow, and the output is the predicted traffic category. As illustrated in Fig. 3, SaGNN consists of four main components. First, the H-TIG Construction module transforms the encrypted flow into an H-TIG at the flow level. Then, the Triple Embedding module performs fine-grained representation at the packet level to avoid semantic conflicts and enrich the node attributes of the H-TIG. Next, the STGAT for Graph Representation module uses STGAT, the carefully designed heterogeneous graph neural network, to hierarchically learn and abstract both the interaction relationships and rich node attributes in the constructed H-TIG, producing a comprehensive flow-level representation vector. Finally, the Classifier maps the flow-level representation vector to the corresponding traffic category. The following subsections introduce these components in detail.

4.1. H-TIG construction

We design a Heterogeneous Traffic Interaction Graph, namely H-TIG, to represent an encrypted flow. H-TIG retains the interaction relationships among packets within a flow while distinguishing the semantic information between the Handshake Stage and the Transmission Stage of encrypted communication. The construction process of H-TIG is shown in Fig. 2. As depicted in Fig. 2(a), each packet exchanged between client and server is represented as a line with an arrow. The arrow indicates the direction of the packet and the color of the line indicates stage to which the packet belongs. Each individual gray block represents a burst, where a burst is defined as a sequence of consecutive packets transmitted along the same direction during a short time interval [45]. Note that a burst may contain only a single packet, just like the burst containing packet a. The encrypted traffic presented in Fig. 2(a) is finally transformed to H-TIG, as depicted in Fig. 2(b). As one kind of heterogeneous graph, H-TIG contains more than one type of vertices or edges.

Vertices: Each packet in the flow is represented as a vertex in H-TIG. Vertices in the Handshake Stage are represented by the solid blue circles, and vertices in the Transmission Stage are represented by the solid orange circles. Each vertex has an attribute f_p, which is a vector characterizing the entire packet. The method for obtaining the attribute of each vertex is described in Section 4.2.

Edges: Edges indicate the relationships between vertices, there are three types of relationships in H-TIG: shake–shake, trans–trans, and shake–trans. Shake–shake describes the internal relationship between packets of the Handshake Stage, trans–trans describes the internal relationship between packets of the Transmission Stage, and shake–trans describes the inter-stage relationship between packets of the Handshake Stage and the Transmission Stage. Both shake–shake and trans–trans relationships correspond to two types of edges: intra-burst edges and inter-burst edges. Intra-burst edges connect all vertices within a burst in chronological order. For instance, three vertices of burst $[d, e, f]$ in the Handshake Stage are connected by two intra-burst edges. Inter-burst edges connect vertices from adjacent bursts. Specifically, the first and last vertices of a subsequent burst are respectively connected to the first and last vertices of the preceding burst. If a burst contains only a single vertex, the vertex is regarded as both the first and the last vertex. For example, the edges between vertex b and d, c and f, h and j are all inter-burst edges. As for shake–trans relationship, it only corresponds to inter-burst edges, such as edges between vertex d and g, f and h. To facilitate information propagation among vertices, all edges are undirected. Note that this design does not obscure packet-direction semantics, since the burst partition is direction-aware and packets with different transmission directions are assigned to different bursts. If directed edges are used, each vertex can only aggregate information from one temporal direction, which limits the propagation of contextual information. In contrast, undirected edges enable each vertex to exchange information with both preceding and succeeding neighboring vertices, thereby preserving more comprehensive interaction patterns within the flow.

The formal definition of H-TIG is given as follows.

Definition 1 (***Heterogeneous Traffic Interaction Graph***)**.** An H-TIG is defined by a quintuple (V, E, F, L, R):

- V is a set of vertices. Each vertex $v \in V$ represents a packet and has two attributes, attribute $f_p \in F$ is a vector characterizing the entire packet and attribute $l_p \in L$ presents type of the packet.
- E is a set of edges. Each edge $e \in E$ either connects vertices within a burst or vertices between two bursts, and each edge has an attribute $r_e \in R$ presenting type of the edge.
- F is the set of vertex attribute f_p. Approach to get f_p is illustrated in subsequent Section 4.2.
- L is the set of vertex attribute l_p, i.e., $L = \{shake, trans\}$
- R is the set of edge attribute r_e, i.e., $R = \{shake - shake, trans - trans, shake - trans\}$

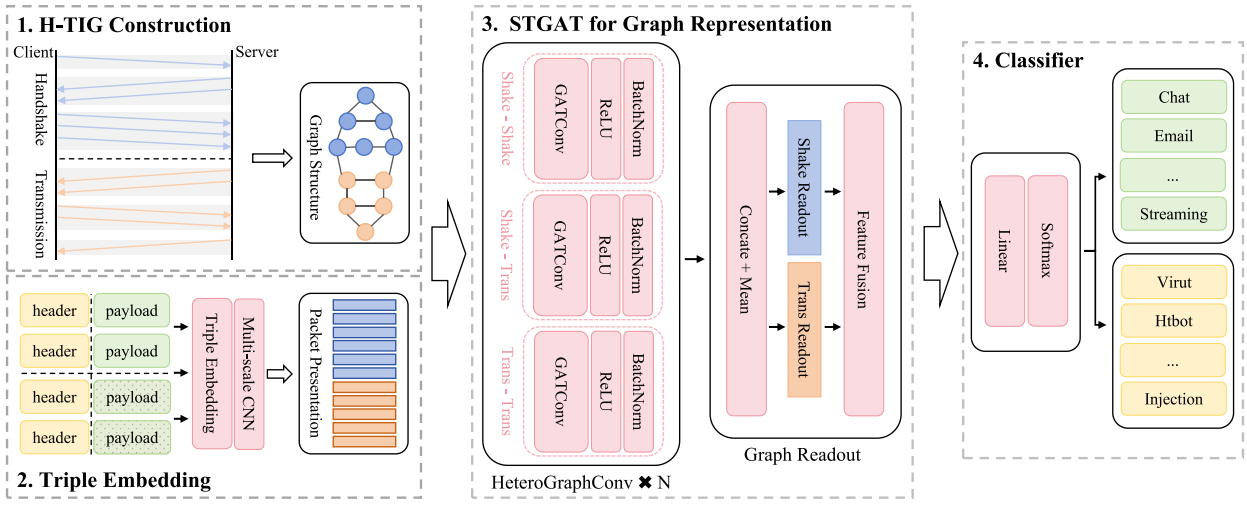


Fig. 3. Workflow of SaGNN.

Algorithm 1 Construction of H-TIG

Input: A flow $P = [p_1, p_2, ..., p_i, ..., p_n]$
Output: The corresponding heterogeneous traffic interaction graph $G = (V, E, L, R)$

1: Initialize $V = \{\}$, $V_{shake} = \{\}$, $V_{trans} = \{\}$, $E = \{\}$
2: $L = \{shake, trans\}$, $R = \{shake - shake, trans - trans, shake - trans\}$
3: Divide P into two lists P_{shake} and P_{trans} according to the stage to which each packet belongs
4: **for** $p_i \in P_{shake}$ **do**
5: Add a vertex with $l_p = shake$, f_p got from Triple Embedding into V_{shake}
6: **end for**
7: **for** $p_j \in P_{trans}$ **do**
8: Add a vertex with $l_p = trans$, f_p got from Triple Embedding into V_{trans}
9: **end for**
10: $V = V_{shake} + V_{trans}$
11: Separate V_{shake} into bursts $B_{shake} = [b_1, ..., b_k]$ according to packet direction and timestamp
12: **for** $b_i \in B_{shake}$ **do**
13: Add intra-burst edges within b_i and inter-burst edges between b_i and b_{i+1} into E
14: Set attribute of edges, $r_e = shake - shake$
15: **end for**
16: **for** $b_i \in B_{trans}$ **do**
17: Add intra-burst edges within b_i and inter-burst edges between b_i and b_{i+1} into E
18: Set attribute of edges, $r_e = trans - trans$
19: **end for**
20: Add inter-burst edges between $B_{shake}[-1]$ and $B_{trans}[1]$
21: Set attribute of edges, $r_e = shake - trans$
22: **return** G

The construction process of H-TIG is formally described in Algorithm 1. Line 1 and 2 initialize the sets V, E, L, and R. Line 3–10 divide the flow into two lists and add two types of vertices into V. Line 11–21 create three types of edges and add them into E. A heterogeneous traffic interaction graph G is returned at last. There are many advantages of H-TIG. On the one hand, by setting vertex attribute f_p, H-TIG contains rich information of packets in an encrypted flow. On the other hand, by constructing intra-burst and inter-burst edges, H-TIG depicts behavioral details such as burst level characteristics and packet sequence information, thus characterizing the interactive features within a flow. Most important of all, by setting vertex attribute l_p, edge attribute r_e, H-TIG naturally differentiates between packets and their interactions in the Handshake Stage and the Transmission Stage, thereby capturing the key characteristic of the change in flow interaction pattern introduced by encryption protocols.

4.2. Triple embedding

To avoid semantic conflicts at the packet level and enrich the node attributes of the H-TIG, we design Triple Embedding to obtain the packet representation vector f_p. The obvious advantage of Triple Embedding is that it achieves fine-grained semantic distinction, as a result of dealing with packet header, packet payload in the Handshake Stage and packet payload in the Transmission Stage separately. As described above, the same byte appearing in different locations across the three parts is likely to carry different semantic meanings, due to different functions and manifestations. Triple Embedding can avoid semantic conflicts and make more accurate representation. Step 2 of Fig. 3 depicts details of Triple Embedding.

Specifically, we set three learnable embedding matrices E_h, E_s, $E_t \in \mathbb{R}^{256 \times D_e}$ for bytes of packet header, packet payload in the Handshake Stage and packet payload in the Transmission Stage separately, where the number of possible integer values of a byte is 256, and D_e is the

embedding dimension of a byte. Initially, the header of a packet is represented as $H_0 \in \mathbb{R}^{N_h \times 256}$, where each byte is denoted as a one-hot vector and N_h is the number of bytes in header. After multiplying with E_h, we get $H_e \in \mathbb{R}^{N_h \times D_e}$ for the header. Here, the one-hot vector is only used as an indexing form rather than a high-dimensional feature directly processed by subsequent modules. In this way, each byte value is mapped to a D_e-dimensional learnable embedding vector, which can encode richer semantic information than the original scalar byte value. Similarly, $S_e \in \mathbb{R}^{N_p \times D_e}$ and $T_e \in \mathbb{R}^{N_p \times D_e}$ are obtained for other two parts, where N_p is the number of bytes in payload.

Inspired by TextCNN [46], we use a similar approach to extract higher-level feature of packet. TextCNN is adept at sentence representation and we treat a byte in packet as a word in sentence. Three convolutional kernels of different scales are employed, with N_{kernel} kernels configured for each scale. The result of processing a packet in the Handshake Stage by convolutional kernel k is a scalar:

$$f_{shake}^{k} = \text{MaxPool}(\text{ReLU}(\text{Conv}([H_e \parallel S_e]))) \tag{2}$$

The concatenated H_e and S_e is processed sequentially by a convolutional layer, the ReLU activation function and a max-pooling layer. After concatenating results from all convolutional kernels, we get

$$f_{shake} = \|_{k=1}^{3 \times N_{kernel}} f_{shake}^{k} \tag{3}$$

Then, we utilize a fully connected layer with the ReLU function to get the final representation of the packet in the Handshake Stage:

$$f_p^{shake} = \text{ReLU}(\text{FC}(f_{shake})) \tag{4}$$

The presentation of the packet in the Transmission Stage f_p^{trans} is obtained by the same operation for concatenated H_e and T_e.

4.3. STGAT for graph representation

We design a heterogeneous graph neural network (HGNN), namely STGAT, to further learn the rich and discriminative semantic features in H-TIG and finally obtain a comprehensive representation. H-TIG contains three types of relationships, each relationship is regarded as a subgraph. In other words, H-TIG is composed of three subgraphs sharing some common vertices. HGNN is specifically used to deal with heterogeneous graphs with complex relationships. As shown in step 3 of Fig. 3, STGAT mainly consists of HeteroGraphConv Block and Graph Readout, which will be introduced below.

(1) HeteroGraphConv Block: HeteroGraphConv Block is a stack of N_l HeteroGraphConv layers, and a HeteroGraphConv layer employs individual graph neural networks followed by the ReLU activation function and BatchNorm operation to process different subgraphs. We employ Graph Attention Network (GAT) for information extraction from the subgraph. By incorporating attention mechanism, GAT weights information from neighboring nodes according to different attention coefficients during message aggregation. This design amplifies the contributions of important information while suppressing less relevant signals, thereby enhancing the efficiency of information extraction.

When processing subgraph r, the attention coefficient of node i from its neighboring node j is calculated by:

$$\alpha_{i,j}^{r} = \frac{\exp\left(\text{LeakyReLU}\left(a^T \left[W h_i \parallel W h_j\right]\right)\right)}{\sum_{k \in \mathcal{N}_i} \exp\left(\text{LeakyReLU}\left(a^T \left[W h_i \parallel W h_k\right]\right)\right)} \tag{5}$$

where h_i and h_j are the feature vectors of nodes i and j, respectively, W is the projection matrix, a is learnable parameter vector, and $\mathcal{N}_i$ is the set of neighbors of node i.

After message aggregation, new feature vector of node i is obtained:

$$h_i^{r,1} = \sigma\left(\sum_{j \in \mathcal{N}_i} \alpha_{ij}^{r} W h_j\right) \tag{6}$$

where σ is the activation function. Additionally, the multi-head attention mechanism is utilized to enhance the learning process. Three GAT modules work concurrently, so new feature vectors of nodes in all subgraphs are obtained simultaneously. We further use the Mean operation to aggregate the feature vectors of the same node across different subgraphs, thereby obtaining a unique representation for each node.

$$h_i^1 = \text{Mean}_{r \in R}\left(h_i^{r,1}\right) \tag{7}$$

(2) Graph Readout: We first concatenate all feature vectors output by N_l HeteroGraphConv layers as representation of node i, then utilize the mean pooling on all nodes to get Shake Readout H_{shake} and Trans Readout H_{trans}:

$$H_s = \frac{1}{N_s} \sum_{i=1}^{N_s} \left(\|_{z=1}^{N_l} h_i^z \right), s \in L \tag{8}$$

where s denotes the Handshake Stage or the Transmission Stage the node belongs to, N_s is number of nodes in the stage.

Finally, we deploy Cross-gate feature fusion [21] on H_{shake} and H_{trans} to get the presentation vector of H-TIG:

$$F_G = \|_{s \in L} \text{Sigmoid}(W_s H_s) \odot H_{L/s} \tag{9}$$

where W_s is learnable parameter, Sigmoid is the activation function, and $\odot$ denotes element-wise product. This fusion mechanism uses one stage representation to generate a gate for the other, helping filter out unimportant information and retain significant complementary features between the Handshake Stage and the Transmission Stage.

4.4. Classifier

H-TIG presentation vector F_G is input to a classifier to output the result of encrypted traffic classification. The classifier consists of two fully connected layers, followed by a ReLU activation function and a Softmax activation function, respectively.

$$\hat{y} = \text{Softmax}\left(W_2 \,\text{ReLU}\left(W_1 F_G\right)\right) \tag{10}$$

where W_1 and W_2 are learnable parameters for fully connected layers, and $\hat{y}$ is the predicted probability vector of the encrypted flow. Cross-entropy is utilized as the loss function for model training.

$$\text{Loss} = -\sum_{i=1}^{|C|} y_i \log \hat{y}_i \tag{11}$$

5. Experiments

In this section, we first present the experimental settings, including selected datasets, data pre-processing, implementation details, baseline methods, and evaluation metrics. We then conduct a series of experiments to comprehensively validate the performance of the proposed SaGNN, including comparative experiments, ablation studies, model sensitivity analysis, and model complexity analysis.

5.1. Experimental settings

5.1.1. Datasets

We adopt multiple publicly available datasets to comprehensively evaluate the performance of the proposed SaGNN, i.e., ISCX VPN-nonVPN [47] and USTC-TFC [48].

ISCX VPN-nonVPN dataset is released by the Canadian Institute for Cybersecurity (CIC). It contains encrypted network traffic generated from a variety of real-world applications, such as VoIP, peer-to-peer (P2P), streaming, and web services. Specifically, ISCX VPN-nonVPN is divided into two subsets: ISCX-VPN and ISCX-nonVPN. ISCX-VPN dataset consists of traffic captured over virtual private networks (VPNs). VPNs are widely used to access blocked websites or services and to protect user privacy, and it is difficult to recognize and classify VPN traffic because of the use of tunneling and obfuscation technologies. In contrast, the ISCX-nonVPN subset contains regular encrypted traffic that is not transmitted through VPN tunnels.

Table 2
Number of remaining flows per class after preprocessing.

Dataset	Class	#Flows
ISCX-nonVPN	Chat	282
	Email	203
	File	310
	Video	254
	Streaming	582
	VoIP	609
ISCX-VPN	Chat	148
	Email	131
	File	138
	P2P	8
	Streaming	241
	VoIP	401
USTC-TFC	Virut(malware)	6171
	Shifu(malware)	9580
	Neris(malware)	8861
	Htbot(malware)	6254
	BiTorrent(benign)	7502
	Gmail(benign)	6442
	Outlook(benign)	7475

USTC-TFC dataset is released by the University of Science and Technology of China (USTC). It contains benign and malicious traffic collected from various applications and malware samples. The benign traffic includes various commonly used applications such as web browsing, email, and file transfer, while the malicious traffic is generated by different categories of malware, including trojans, botnets, and other malicious programs. This dataset is adopted to demonstrate the ability of the proposed method in encrypted traffic classification and malicious traffic detection.

5.1.2. Pre-processing and implementation details

The raw network traffic in the dataset is provided in PCAP (Packet Capture) format, which requires pre-processing to transform it into a format suitable for model input. The pre-processing pipeline consists of the following steps:

(1) First, the original traffic is segmented into bidirectional flows using the SplitCap tool. The segmentation is based on the five-tuple (source IP, destination IP, source port, destination port, and transport protocol). (2) Subsequently, traffic filtering is applied to remove packets with empty payload, as such packets typically serve to establish basic connections or convey simple acknowledgment (ACK) messages, and thus do not contribute meaningfully to encrypted traffic classification. Additionally, retransmitted packets, duplicated packets, and malformed or corrupted packets that cannot be correctly parsed are excluded. Extremely long flows and empty flows resulting from packet filtering are also removed. (3) The packets are then processed further: header fields of Ethernet, IP and port are stripped, as these fields primarily facilitate basic networking functions and their retention may bias the model toward less discriminative features. To standardize packet length, truncation and padding are applied. For both packet headers and payloads, bytes exceeding the predefined lengths are truncated, while missing bytes are padded with the value 256 rather than 0, as the value 0 occurs naturally in raw packet data; padding with 0 could otherwise introduce ambiguity. (4) Finally, the processed traffic is structured into graph-based representations to serve as input for downstream model training and evaluation.

Since this work focuses on encrypted traffic classification, dataset-specific filtering is applied. For ISCX VPN-nonVPN, flows without encrypted packets are removed while all categories are retained. For USTC-TFC, categories containing encrypted flows are retained to evaluate classification when encrypted and non-encrypted flows coexist. Table 2 reports the remaining flows per dataset and class.

For H-TIG construction, we set the maximum number of packets in a flow to 30. The byte length of packet header N_h and the byte length of payload N_p are set to 40 and 300, respectively. The embedding dimension of a byte D_e is set to 64, the number of attention heads and HeteroGraphConv layers N_l are both set to 4. Three convolutional kernel scales are set to 2, 3, and 4, with N_{kernel} set to 32 for each scale. These hyperparameters are determined by combining experimental comparison with commonly used settings in related studies, while considering both classification performance and model complexity. In the training phase, we set the max training epoch to 120 and set the batch size to 64. The initial learning rate is set to 0.01 and we use the Adam optimizer with a learning rate scheduler that decays the learning rate gradually.

All datasets are partitioned with a stratified 9:1 split for model training and testing, and all models are implemented with PyTorch on a single NVIDIA H100 GPU. The baseline methods are trained and tested under the same settings as the proposed method, including the same preprocessed datasets, data split strategy, and hardware platform.

5.1.3. Baselines and evaluation metrics

We use seven representative methods as baselines, including three traditional deep learning models and four GNN models:

(1) BiLSTM-Att [35]: BiLSTM-Att adopts BiLSTM to learn raw byte feature and gets presentation of the flow based on attention mechanism.

(2) TSCRNN [34]: TSCRNN combines CNN and stacked BiLSTM to jointly learn spatial and temporal features from encrypted traffic flows for fine-grained traffic classification.

(3) App-Net [49]: App-Net uses LSTM to learn sequence feature of packet lengths and uses CNN to learn payload byte feature of the initial packet concurrently.

(4) GraphDApp [39]: GraphDApp constructs traffic interaction graph of a flow, and use packet length and direction as node feature to achieve traffic classification.

(5) DE-GNN [23]: DE-GNN also constructs traffic interaction graph, and encodes packet header and payload as node feature separately.

(6) TFE-GNN [21]: TFE-GNN presents packet as a graph, which is constructed based on PMI value of bytes, and uses LSTM for downstream classification task.

(7) MH-Net* [22]: MH-Net constructs multi-view heterogeneous graphs at the packet level and performs flow classification using an RNN. MH-Net* denotes our partial reimplementation of MH-Net, focusing on its graph construction strategy without incorporating the contrastive learning and multi-task learning objectives in the original MH-Net training pipeline.

Four metrics are adopted for evaluation: Accuracy (AC), Recall (RE), Precision (PR), and F1 Score (F1), which are calculated based on the number of true positive (TP), false positive (FP), true negative (TN), and false negative (FN). Accuracy is the ratio of correctly predicted instances to the total instances, Precision is the ratio of correctly predicted positive instances to all instances predicted as positive, Recall is the ratio of correctly predicted positive instances to all actual positive instances, and F1 Score is the harmonic mean of Precision and Recall. In addition, macro-averaged metrics are reported to reduce the influence of class imbalance. The formal definitions of the metrics are as follows:

$$AC = \frac{TP + TN}{TP + TN + FP + FN} \tag{12}$$

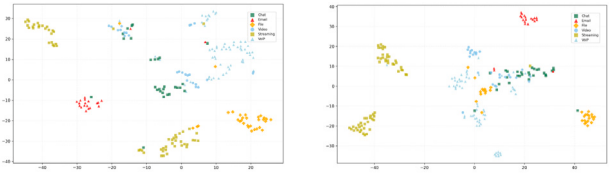
$$PR = \frac{TP}{TP + FP} \tag{13}$$

$$RE = \frac{TP}{TP + FN} \tag{14}$$

$$F1 = 2 \times \frac{PR \times RE}{PR + RE} \tag{15}$$

Table 3
Comparison results on ISCX-VPN, ISCX-nonVPN and USTC-TFC datasets.

Dataset	ISCX-VPN				ISCX-nonVPN				USTC-TFC			
Method	AC	PR	RE	F1	AC	PR	RE	F1	AC	PR	RE	F1
BiLSTM-Att	0.8972	0.9098	0.8986	0.9027	0.7098	0.7103	0.6898	0.6926	0.8290	0.8664	0.8246	0.8154
TSCRNN	0.9439	0.9500	0.9427	0.9457	0.7321	0.7141	0.7018	0.7002	0.9767	0.9755	0.9768	0.9761
App-Net	0.9537	0.9782	0.9486	0.9597	0.7857	0.7704	0.7308	0.7300	0.9686	0.9689	0.9657	0.9667
GraphDApp	0.5392	0.4617	0.4770	0.4450	0.4633	0.4432	0.4468	0.4240	0.5281	0.5836	0.5054	0.5166
DE-GNN	0.9346	0.9329	0.9222	0.9272	0.8795	0.8621	0.8638	0.8625	0.9901	0.9892	0.9906	0.9899
TFE-GNN	0.9626	0.9765	0.9589	0.9670	0.8080	0.7894	0.7811	0.7825	0.9901	0.9888	0.9910	0.9897
MH-Net*	0.9533	0.9514	0.9548	0.9528	0.7634	0.8123	0.7126	0.7359	0.9914	0.9914	0.9910	0.9912
SaGNN	**0.9907**	**0.9959**	**0.9931**	**0.9944**	**0.9107**	**0.8994**	**0.9051**	**0.8994**	**0.9952**	**0.9950**	**0.9951**	**0.9951**



(a) Presentation with SaGNN. (b) Presentation with DE-GNN.

Fig. 4. t-SNE visualization on ISCX-nonVPN.

Table 4
Ablation studies on ISCX-nonVPN and USTC-TFC datasets (Top: ISCX-nonVPN, Bottom: USTC-TFC).

Method	Accuracy	Precision	Recall	F1 score
w/o TE	0.8348	0.8130	0.7944	0.8007
	0.9922	0.9917	0.9924	0.9920
w/ DE	0.8661	0.8499	0.8462	0.8473
	0.9935	0.9931	0.9936	0.9933
w/o HG	0.8884	0.8805	0.8684	0.8737
	0.9948	0.9948	0.9947	0.9947
w/ PO	0.8884	0.8731	0.8703	0.8693
	0.9946	0.9944	0.9946	0.9945
SaGNN	**0.9107**	**0.8994**	**0.9051**	**0.8994**
	0.9952	**0.9950**	**0.9951**	**0.9951**

5.2. Comparison experiments

The results of comparison experiments are shown in Table 3, we can draw some important conclusions from the results: (1) The proposed SaGNN achieves the best performance on four metrics compared with seven implemented baselines across all three datasets, demonstrating its exceptional effectiveness and stability in encrypted traffic classification. (2) The results on USTC-TFC dataset also confirms the satisfactory ability of SaGNN in encrypted malicious traffic detection, SaGNN can not only detect malicious traffic from benign traffic but also distinguish specific types of malicious traffic with high Accuracy and Recall. (3) All methods perform better on ISCX-VPN than on ISCX-nonVPN, indicating that VPN traffic may actually exhibit more distinctive features for classification, despite the use of tunneling and obfuscation technologies. (4) For deep learning based methods, BiLSTM-Att achieves relatively lower accuracy, whereas TSCRNN and App-Net exhibit comparable performance. BiLSTM-Att only captures temporal information of flow by leveraging BiLSTM, while TSCRNN and App-Net capture both spatial and temporal information, resulting in comprehensive representation. App-Net performs better than TSCRNN on ISCX-VPN and ISCX-nonVPN datasets, because App-Net adopts embeddings of bytes as input rather than raw bytes, which enables richer semantic representation and improves robustness to noise. (5) For graph neural network based methods, GraphDApp does not yield noteworthy outcomes on three datasets, because it only uses packet length and direction as node feature, which is susceptible to environmental interference, such as the adoption of packet fragmentation, VPN and software update. Although the other three GNN-based methods exhibit competitive performance, they fail to distinguish between the Handshake Stage and the Transmission Stage of encrypted traffic, overlooking the richer semantic information introduced by encryption protocols, and thus their performance falls short of the proposed SaGNN.

Visualization analysis. To provide a more intuitive understanding of the representations learned by SaGNN, we further visualize the flow-level representation vectors of SaGNN and DE-GNN on the ISCX-nonVPN dataset using t-SNE, as shown in Fig. 4. Compared with DE-GNN, the representations learned by SaGNN exhibit more compact intra-class distributions and clearer inter-class boundaries. For example, several traffic categories that are partially mixed in DE-GNN become more distinguishable in the representation space of SaGNN. This indicates that the proposed H-TIG construction and Triple Embedding help SaGNN learn more discriminative semantic representations, thereby providing representation-level interpretability for the classification results. It should be noted that this t-SNE visualization provides representation-level evidence rather than a direct mechanism-level explanation. Directly interpreting the local multi-head attention coefficients in STGAT as global node or edge importance across different stages may be misleading. More faithful mechanism-level visualization will be explored in future work.

5.3. Ablation studies

To assess the contribution of each component or operation in SaGNN, we conduct ablation studies on ISCX-nonVPN and USTC-TFC, the results are shown in Table 4. To facilitate the presentation of results, we denote Triple Embedding, Dual Embedding, Heterogeneous Graph, and Padding with 0 as TE, DE, HG and PO respectively. Dual Embedding means embedding the bytes in header and payload separately, it is a kind of alternative module of proposed Triple Embedding. Specifically, "w/o TE" denotes a SaGNN variant in which Triple Embedding is replaced with unified embedding, and "w/o HG" denotes another SaGNN variant in which H-TIG is replaced with a homogeneous graph.

The following key points can be observed from Table 4: (1) When eliminating Triple Embedding modules, each byte in all packets is dealt in the same way. The F1 score declines to varying degrees on the two datasets, it declines by 9.87% on the ISCX-nonVPN dataset and by 0.31% on the USTC-TFC dataset. When replacing Triple Embedding with Dual Embedding, F1 score decreases by 5.21% and 0.18%. This indicates the fine-grained semantic perception capability of Triple Embedding, which introduces stage-aware semantic information into node attributes by separately encoding the packet header, handshake payload, and transmission payload. (2) The designs of H-TIG and STGAT for Graph Representation achieve an F1-score improvement of up to 2.57% across the two datasets, which demonstrates their effectiveness in classification. Different from Triple Embedding, which introduces stage-aware semantics into packet-level node attributes, H-TIG models stage-aware packet interactions at the flow level, showing a less pronounced gain on the tested datasets. This may be because

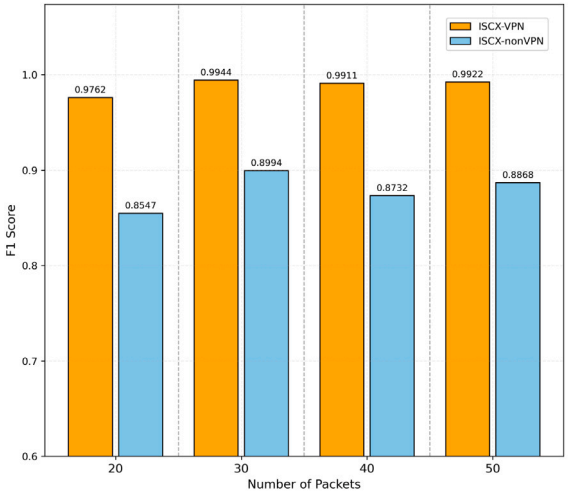


Fig. 5. The impact of packet number.

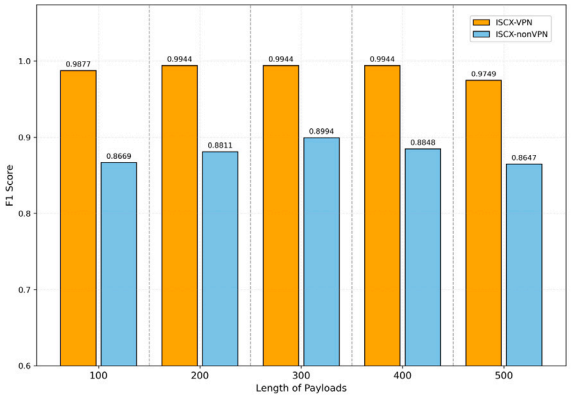


Fig. 6. The impact of payload length.

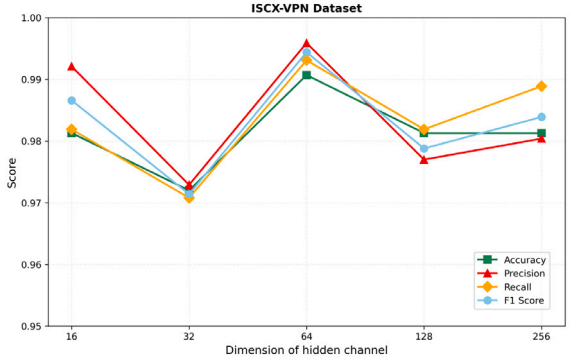


Fig. 7. The impact of hidden dimension on ISCX-VPN.

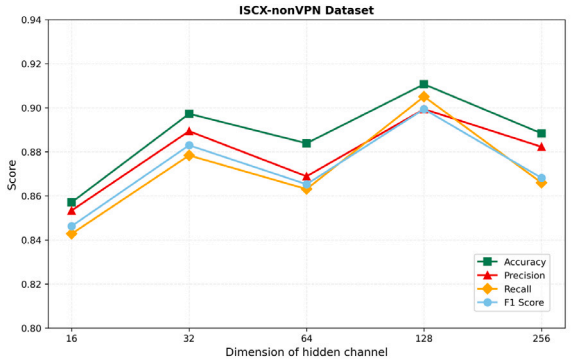


Fig. 8. The impact of hidden dimension on ISCX-nonVPN.

packet-level semantic conflicts have a more direct impact on representation learning, while the performance on USTC-TFC is already close to saturation. (3) We further explore the impact of the padding method on the model. When padding with 0 instead of 256, the F1 score decreases by 1.54% on average, demonstrating that padding with 256 effectively mitigates potential semantic ambiguity. The byte values of packets range from 0 to 255. During the initialization of the embedding layer, if the missing bytes are padded with 256, the padded bytes and the original packet bytes are mapped to different embedding vectors, enabling the model to distinguish between these two types of bytes. In contrast, if 0 is used for padding, the padded bytes and the original bytes with value 0 in packets are mapped to the same embedding vector, which causes semantic ambiguity and ultimately degrades the classification performance of the model.

5.4. Model sensitivity analysis

We analyze the impacts of four major hyperparameters in SaGNN on classification performance. These hyperparameters include packet number, payload length, hidden dimension, and burst interval. The results are presented in Figs. 5–9 respectively.

The impact of the number of packets on model performance is illustrated in Fig. 5. The number of packets within a flow directly determines the node scale of H-TIG. We start from 20 packets because an excessively small number of packets would cause the flow to contain only information from the Handshake Stage of the encrypted traffic. As the number of packets increases, the fluctuation of the F1 score on the ISCX-nonVPN dataset is more pronounced than that on the ISCX-VPN dataset. When the number of packets increases from 20 to 30, the flow incorporates more packets from the Transmission Stage, leading to a better balance of information between the Handshake Stage and the Transmission Stage. Consequently, the F1 scores on both datasets exhibit a significant improvement. As the number of packets continues to increase, the F1 scores on both datasets decline to varying degrees. This is because the additional packets in the Transmission Stage weaken the influence of the packets in the Handshake Stage. Although retaining more packets in each flow may help capture the patterns contained in long-lived flows, it is also likely to introduce noise into the representations of other flows, thereby degrading the overall classification performance. Therefore, we set the number of packets per flow to 30.

For packets, the header length is relatively stable, we fix it to 40. Then we conduct a series of experiments to investigate the impact of payload length variation, with the results shown in Fig. 6. The payload length affects node representations and, consequently, the representation capability of H-TIG. Overall, as the payload length increases, the F1 scores of SaGNN on both datasets exhibit a trend of first increasing and then decreasing. For the ISCX-VPN dataset, the F1 score remains at a relatively high and stable level when the payload length ranges from 200 to 400. For the ISCX-nonVPN dataset, the F1 score reaches its peak when the payload length is 300. When the payload length is short, effective information contained in the subsequent payload is ignored, resulting in insufficient feature extraction. In contrast, when the payload length is excessively long, redundant data introduce a certain amount of noise that interferes with model inference and also incurs higher computational cost. Considering these factors, we set the payload length of each packet to 300.

We further investigate how the hidden-layer dimensionality affects model performance. In general, a larger hidden dimension enhances the

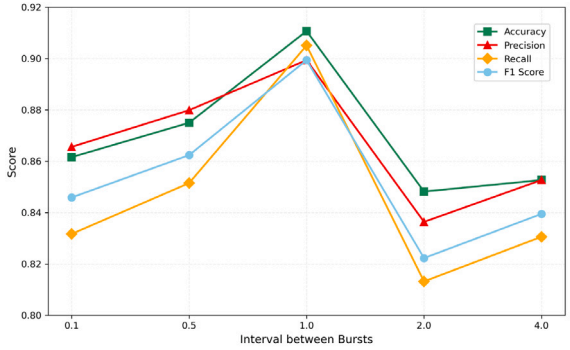


Fig. 9. The impact of burst interval on ISCX-nonVPN.

model's representational capacity, at the cost of higher computational overhead. Figs. 6 and 7 illustrate the variations of all evaluation metrics on the ISCX-VPN and ISCX-nonVPN datasets, respectively. Overall, the metrics tend to improve as the hidden dimension increases, and this trend is more pronounced on ISCX-nonVPN. Nevertheless, increasing the hidden dimension does not monotonically improve performance. On ISCX-VPN, all metrics peak when the hidden dimension is 64, whereas on ISCX-nonVPN the best performance is achieved with a hidden dimension of 128. Further increasing the hidden dimension may lead to overfitting and consequently degrade performance. Therefore, an appropriate hidden dimension should be selected by considering the dataset scale and distribution, along with the associated costs in computation, storage, and latency. Despite these variations, the metrics remain within narrow fluctuation ranges (1.87%–2.30% on ISCX-VPN dataset and 4.60%–6.23% on ISCX-nonVPN dataset), indicating the robustness and stability of the proposed model.

Since the construction of H-TIG relies on the key concept of burst, we also analyze the impact of the burst interval on model performance. The burst interval denotes the minimum temporal gap used to separate two adjacent bursts with the same packet direction. It directly affects burst partitioning and consequently influences the construction of H-TIG. Fig. 9 shows the performance variation of SaGNN under different burst intervals on the ISCX-nonVPN dataset. When the burst interval is set to 1 s, SaGNN achieves the best classification performance, and thus we adopt this setting. When the burst interval is too small, the number of bursts increases and each burst contains fewer packets. In this case, the graph structure gradually degenerates toward sequential modeling, leading to degraded classification performance. In contrast, when the burst interval becomes too large, the number of bursts decreases, and a single burst may contain packets that share the same direction but differ in function and content. Such coarse-grained burst partitioning also weakens the representation capability of H-TIG, thereby reducing the classification performance.

5.5. Model robustness analysis

To evaluate the robustness of SaGNN under adverse network conditions, we conduct controlled perturbation experiments on the USTC-TFC dataset. The model is trained on the original training set and evaluated on perturbed test flows. Specifically, we simulate packet loss by randomly dropping 5%, 10%, and 20% of packets in each flow. We simulate packet reordering by randomly swapping adjacent packet pairs with a probability of 0.5, and simulate delay jitter by adding Gaussian noise with a mean of 0 s and a standard deviation of 0.1 s to the inter-packet time intervals before H-TIG construction. The unperturbed test setting is denoted as None. The results are summarized in Table 5.

The results in Table 5 show that SaGNN maintains stable performance under moderate perturbations. When 5% and 10% of packets are dropped, the F1 score decreases by only 0.34% and 0.67%, from 0.9951 to 0.9917 and 0.9884, respectively. Under the more severe 20% packet dropping setting, the F1 score decreases by 2.73%, reaching 0.9678, indicating that excessive packet loss may remove important heterogeneous interaction relationships and byte content information, thereby weakening the overall flow representation. In contrast, packet swapping and delay jitter have relatively limited impact, with F1 scores of 0.9925 and 0.9949, respectively. This may be because H-TIG captures complex heterogeneous packet interactions beyond simple sequential features, while Triple Embedding mainly exploits byte content information, which is less sensitive to time-related network dynamics. These results demonstrate that SaGNN shows satisfactory robustness under common network perturbations, while packet loss remains the most influential adverse factor.

Table 5
Robustness performance under different perturbation settings on USTC-TFC.

Perturbation	AC	PR	RE	F1
Drop (5%)	0.9922	0.9918	0.9916	0.9917
Drop (10%)	0.9891	0.9890	0.9878	0.9884
Drop (20%)	0.9699	0.9720	0.9652	0.9678
Swap	0.9929	0.9926	0.9924	0.9925
Jitter	0.9950	0.9948	0.9950	0.9949
None	0.9952	0.9950	0.9951	0.9951

Table 6
Model parameters and inference time.

Method	Parameters (M)	Time (ms)
BiLSTM-Att	1.55e+0	1.15e+2
TSCRNN	2.89e+0	1.81e+2
App-Net	2.61e+0	1.57e+2
GraphDApp	2.00e−1	7.11e+1
DE-GNN	1.98e+0	9.67e+2
TFE-GNN	3.33e+1	8.98e+2
MH-Net*	1.93e+2	1.76e+3
SaGNN	2.50e+0	1.57e+3

5.6. Model complexity analysis

To comprehensively evaluate the trade-off between classification performance and model complexity, we analyze the proposed SaGNN and all baseline methods in terms of the number of model parameters and the inference time for classifying a single encrypted flow. The results are reported in Table 6.

From Tables 3 and 6, we can observe that SaGNN achieves the best overall performance on the three public datasets, while introducing only a slight increase in model complexity. Deep learning-based methods such as BiLSTM-Att simply model encrypted flows as images or sequences, and therefore have relatively fewer parameters and lower inference time. However, their classification performance is generally inferior. Among GNN-based methods, SaGNN involves more sophisticated heterogeneous graph modeling, resulting in approximately 1.3 times the number of parameters of DE-GNN. Nevertheless, its parameter size remains significantly smaller than those of TFE-GNN and MH-Net*, which also adopt heterogeneous graph modeling. Although MH-Net* achieves performance close to that of SaGNN on the USTC-TFC dataset, SaGNN further reduces the inference time by about 10%. Therefore, SaGNN achieves satisfactory efficiency while maintaining superior classification performance, which is important for practical encrypted traffic classification scenarios.

6. Discussion

Scalability of multi-flow modeling: SaGNN considers only information from a single flow, whereas real-world network flows may

exhibit complex contextual relationships with other flows. Extending the current per-flow framework to large-scale multi-flow modeling may introduce additional computational and memory overhead for constructing and maintaining massive flow graphs and inter-flow relationships. Therefore, future work will explore graph sampling, incremental graph updating, and lightweight inter-flow relation modeling for multi-flow scenarios.

Applicability to emerging protocols: SaGNN relies on the stage distinction between the Handshake Stage and the Transmission Stage in SSL/TLS traffic. For protocols such as QUIC, where handshake and data transmission are more tightly integrated, or TLS session resumption, where the handshake stage is shortened, the stage boundary may become less clear. Extending SaGNN to such scenarios will be investigated in future work.

Robustness and generalization in realistic environments: The controlled perturbation experiments show that SaGNN maintains satisfactory robustness under packet loss, packet reordering, and delay jitter. Nevertheless, real-world encrypted traffic may still involve more complex and heterogeneous network conditions, which may affect packet sequences, burst structures, and interaction patterns. Therefore, evaluation on more diverse real-world datasets and further robustness improvement in complex deployment environments remain important directions for future work.

Parameter selection for practical deployment: The optimal settings of parameters such as payload length may vary across datasets due to different traffic patterns, application types, and packet length distributions. For new deployment scenarios, these parameters can be selected through lightweight evaluation on representative samples or historical traffic. Future work may further explore adaptive truncation or dynamic-length modeling to improve deployment flexibility.

7. Conclusion

In this paper, we propose an encrypted traffic classification method with semantic-aware heterogeneous graph neural network, namely SaGNN, which fully exploits the intrinsic semantic characteristics of encrypted traffic. Specifically, encrypted flow is modeled as Heterogeneous Traffic Interaction Graph (H-TIG), which preserves internal interaction relationships while explicitly distinguishing the Handshake Stage and the Transmission Stage with different semantics. Then, Triple Embedding is designed to independently and accurately encode bytes from different regions of packets, thus avoiding semantic conflicts and enriching the node attributes of the H-TIG. Furthermore, we develop a heterogeneous graph neural network called STGAT to extract rich and discriminative features from H-TIG, enabling accurate and efficient encrypted traffic classification. Extensive experiments on the ISCX VPN-nonVPN and USTC-TFC datasets demonstrate the effectiveness of SaGNN and indicate its potential for practical applications, including encrypted traffic identification for network efficiency optimization and malicious encrypted traffic detection for security monitoring. Considering the dual-use nature of encrypted traffic classifiers, SaGNN should be deployed only in authorized network management and security monitoring scenarios.

CRediT authorship contribution statement

Junjie Luo: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Min Liu:** Writing – review & editing, Supervision, Resources, Project administration. **Sheng Sun:** Writing – review & editing, Supervision. **Yali Chen:** Writing – review & editing, Supervision. **Qixun Tang:** Validation, Software. **Yuwei Wang:** Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the National Natural Science Foundation of China under Grant 62472410.

Data availability

Data will be made available on request.

References

[1] A. Freier, P. Karlton, P. Kocher, The Secure Sockets Layer (SSL) Protocol Version 3.0, Technical Report, 2011.
[2] T. Dierks, E. Rescorla, The Transport Layer Security (TLS) Protocol Version 1.2, Technical Report, 2008.
[3] C. Oh, J. Ha, H. Roh, A survey on TLS-encrypted malware network traffic analysis applicable to security operations centers, Appl. Sci. 12 (1) (2021) 155.
[4] E. Papadogiannaki, S. Ioannidis, A survey on encrypted network traffic analysis applications, techniques, and countermeasures, ACM Comput. Surv. 54 (6) (2021) 1–35.
[5] Y. Liu, X. Wang, B. Qu, F. Zhao, ATVITSC: A novel encrypted traffic classification method based on deep learning, IEEE Trans. Inf. Forensics Secur. (2024).
[6] A. Saber, B. Fergani, M. Abbas, Encrypted traffic classification: Combining over- and under-sampling through a pca-svm, in: 2018 3rd International Conference on Pattern Analysis and Intelligent Systems, PAIS, IEEE, 2018, pp. 1–5.
[7] C. Liu, Z. Cao, G. Xiong, G. Gou, S.-M. Yiu, L. He, Mampf: Encrypted traffic classification based on multi-attribute markov probability fingerprints, in: 2018 IEEE/ACM 26th International Symposium on Quality of Service, IWQoS, IEEE, 2018, pp. 1–10.
[8] M. Shen, Y. Liu, L. Zhu, X. Du, J. Hu, Fine-grained webpage fingerprinting using only packet length information of encrypted traffic, IEEE Trans. Inf. Forensics Secur. 16 (2020) 2046–2059.
[9] S.-J. Xu, G.-G. Geng, X.-B. Jin, D.-J. Liu, J. Weng, Seeing traffic paths: Encrypted traffic classification with path signature features, IEEE Trans. Inf. Forensics Secur. 17 (2022) 2166–2181.
[10] P. Zhang, F. He, H. Zhang, J. Hu, X. Huang, J. Wang, X. Yin, H. Zhu, Y. Li, Real-time malicious traffic detection with online isolation forest over sd-wan, IEEE Trans. Inf. Forensics Secur. 18 (2023) 2076–2090.
[11] N. Wickramasinghe, A. Shaghaghi, E. Ferrari, S. Jha, Less is more: Simplifying network traffic classification leveraging RFCs, in: Companion Proceedings of the ACM on Web Conference 2025, 2025, pp. 1398–1401.
[12] T. Shapira, Y. Shavitt, FlowPic: A generic representation for encrypted traffic classification and applications identification, IEEE Trans. Netw. Serv. Manag. 18 (2) (2021) 1218–1232.
[13] M. Lotfollahi, M. Jafari Siavoshani, R. Shirali Hossein Zade, M. Saberian, Deep packet: A novel approach for encrypted traffic classification using deep learning, Soft Comput. 24 (3) (2020) 1999–2012.
[14] S. Zhu, X. Xu, H. Gao, F. Xiao, CMTSNN: A deep learning model for multiclassification of abnormal and encrypted traffic of Internet of Things, IEEE Internet Things J. 10 (13) (2023) 11773–11791.
[15] M. Shen, J. Wu, K. Ye, K. Xu, G. Xiong, L. Zhu, Robust detection of malicious encrypted traffic via contrastive learning, IEEE Trans. Inf. Forensics Secur. (2025).
[16] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, J. Yu, Et-bert: A contextualized datagram representation with pre-training transformers for encrypted traffic classification, in: Proceedings of the ACM Web Conference 2022, 2022, pp. 633–642.
[17] R. Zhao, M. Zhan, X. Deng, Y. Wang, Y. Wang, G. Gui, Z. Xue, Yet another traffic classifier: A masked autoencoder based traffic transformer with multi-level flow representation, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 37, 2023, pp. 5420–5427.
[18] R. Zhao, M. Zhan, X. Deng, F. Li, Y. Wang, Y. Wang, G. Gui, Z. Xue, A novel self-supervised framework based on masked autoencoder for traffic classification, IEEE/ACM Trans. Netw. 32 (3) (2024) 2012–2025.
[19] G. Zhou, X. Guo, Z. Liu, T. Li, Q. Li, K. Xu, Trafficformer: An efficient pre-trained model for traffic data, in: 2025 IEEE Symposium on Security and Privacy, SP, IEEE, 2025, pp. 1844–1860.
[20] J. Yin, L. Cui, Z. Hao, J. Sun, P. Liu, X. Yun, BTRFormer: Hierarchical learning of encrypted traffic using a masked autoencoder with block-based traffic representation, in: 2025 IEEE 33rd International Conference on Network Protocols, ICNP, IEEE, 2025, pp. 1–12.
[21] H. Zhang, L. Yu, X. Xiao, Q. Li, F. Mercaldo, X. Luo, Q. Liu, TFE-GNN: A temporal fusion encoder using graph neural networks for fine-grained encrypted traffic classification, in: Proceedings of the ACM Web Conference 2023, 2023, pp. 2066–2075.
[22] H. Zhang, H. Yue, X. Xiao, L. Yu, Q. Li, Z. Ling, Y. Zhang, Revolutionizing encrypted traffic classification with MH-net: A multi-view heterogeneous graph model, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 39, 2025, pp. 1048–1056.

[23] X. Han, G. Xu, M. Zhang, Z. Yang, Z. Yu, W. Huang, C. Meng, DE-GNN: Dual embedding with graph neural network for fine-grained encrypted traffic classification, Comput. Netw. 245 (2024) 110372.
[24] S. Cui, X. Han, D. Han, Z. Wang, W. Wang, B. Jiang, B. Liu, Z. Lu, FG-SAT: Efficient flow graph for encrypted traffic classification under environment shifts, IEEE Trans. Inf. Forensics Secur. (2025).
[25] H. Xu, C. Si, S. Li, Z. Cheng, C. Wang, J. Xie, P. Sun, Q. Liu, Flowminer: A powerful model based on flow correlation mining for encrypted traffic classification, in: IEEE INFOCOM 2025-IEEE Conference on Computer Communications, IEEE, 2025, pp. 1–10.
[26] T.-L. Huoh, Y. Luo, T. Zhang, Encrypted network traffic classification using a geometric learning model, in: 2021 IFIP/IEEE International Symposium on Integrated Network Management, IM, IEEE, 2021, pp. 376–383.
[27] S. Rezaei, X. Liu, Deep learning for encrypted traffic classification: An overview, IEEE Commun. Mag. 57 (2019) 76–81.
[28] V.F. Taylor, R. Spolaor, M. Conti, I. Martinovic, Appscanner: Automatic fingerprinting of smartphone apps from encrypted network traffic, in: 2016 IEEE European Symposium on Security and Privacy, EuroS&P, IEEE, 2016, pp. 439–454.
[29] F. Zaki, F. Afifi, S. Abd Razak, A. Gani, N.B. Anuar, GRAIN: Granular multi-label encrypted traffic classification using classifier chain, Comput. Netw. 213 (2022) 109084.
[30] W. Wang, M. Zhu, J. Wang, X. Zeng, Z. Yang, End-to-end encrypted traffic classification with one-dimensional convolution neural networks, in: 2017 IEEE International Conference on Intelligence and Security Informatics, ISI, IEEE, 2017, pp. 43–48.
[31] C. Liu, L. He, G. Xiong, Z. Cao, Z. Li, Fs-net: A flow sequence network for encrypted traffic classification, in: IEEE INFOCOM 2019-IEEE Conference on Computer Communications, IEEE, 2019, pp. 1171–1179.
[32] G. Aceto, D. Ciuonzo, A. Montieri, A. Pescapè, MIMETIC: Mobile encrypted traffic classification using multimodal deep learning, Comput. Netw. 165 (2019) 106944.
[33] X. Wang, S. Chen, J. Su, App-net: A hybrid neural network for encrypted mobile traffic classification, in: IEEE INFOCOM 2020-IEEE Conference on Computer Communications Workshops, INFOCOM WKSHPS, IEEE, 2020, pp. 424–429.
[34] K. Lin, X. Xu, H. Gao, TSCRNN: A novel classification scheme of encrypted traffic based on flow spatiotemporal features for efficient management of IIoT, Comput. Netw. 190 (2021) 107974.
[35] H. Yao, C. Liu, P. Zhang, S. Wu, C. Jiang, S. Yu, Identification of encrypted traffic through attention mechanism based long short term memory, IEEE Trans. Big Data 8 (1) (2019) 241–252.
[36] X. Liu, J. You, Y. Wu, T. Li, L. Li, Z. Zhang, J. Ge, Attention-based bidirectional GRU networks for efficient HTTPS traffic classification, Inform. Sci. 541 (2020) 297–315.
[37] H. Kong, T. Li, J. Ge, H. Li, L. Yang, Z. Lu, Z. Xu, C. Xie, A novel method with transformers for fine-grained encrypted traffic classification, in: 2023 IEEE International Conference on High Performance Computing & Communications, Data Science & Systems, Smart City & Dependability in Sensor, Cloud & Big Data Systems & Application (HPCC/DSS/SmartCity/DependSys), IEEE, 2023, pp. 74–81.
[38] R. Xie, Y. Wang, J. Cao, E. Dong, M. Xu, K. Sun, Q. Li, L. Shen, M. Zhang, Rosetta: Enabling robust tls encrypted traffic classification in diverse network environments with tcp-aware traffic augmentation, in: Proceedings of the ACM Turing Award Celebration Conference-China 2023, 2023, pp. 131–132.
[39] M. Shen, J. Zhang, L. Zhu, K. Xu, X. Du, Accurate decentralized application identification via encrypted traffic analysis using graph neural networks, IEEE Trans. Inf. Forensics Secur. 16 (2021) 2367–2380.
[40] T.-L. Huoh, Y. Luo, P. Li, T. Zhang, Flow-based encrypted network traffic classification with graph neural networks, IEEE Trans. Netw. Serv. Manag. 20 (2) (2022) 1224–1237.
[41] J. Zheng, Z. Zeng, T. Feng, GCN-ETA: High-efficiency encrypted malicious traffic detection, Secur. Commun. Networks 2022 (1) (2022) 4274139.
[42] J. Lu, G. Gou, M. Su, D. Song, C. Liu, C. Yang, Y. Guan, GAP-WF: Graph attention pooling network for fine-grained SSL/TLS website fingerprinting, in: 2021 International Joint Conference on Neural Networks, IJCNN, IEEE, 2021, pp. 1–8.
[43] Z. Diao, G. Xie, X. Wang, R. Ren, X. Meng, G. Zhang, K. Xie, M. Qiao, EC-GCN: A encrypted traffic classification framework based on multi-scale graph convolution networks, Comput. Netw. 224 (2023) 109614.
[44] X. Pei, X. Deng, S. Tian, P. Jiang, Y. Zhao, K. Xue, A privacy-preserving graph neural network for network intrusion detection, IEEE Trans. Dependable Secur. Comput. 22 (2025) 740–756.
[45] K. Al-Naami, S. Chandra, A. Mustafa, L. Khan, Z. Lin, K. Hamlen, B. Thuraisingham, Adaptive encrypted traffic fingerprinting with bi-directional dependence, in: Proceedings of the 32nd Annual Conference on Computer Security Applications, 2016, pp. 177–188.
[46] Y. Kim, Convolutional neural networks for sentence classification, in: Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2014, pp. 1746–1751.
[47] G. Draper-Gil, A.H. Lashkari, M.S.I. Mamun, A. A. Ghorbani, Characterization of encrypted and VPN traffic using time-related features:, in: Proceedings of the 2nd International Conference on Information Systems Security and Privacy, 2016, pp. 407–414.
[48] W. Wang, M. Zhu, X. Zeng, X. Ye, Y. Sheng, Malware traffic classification using convolutional neural network for representation learning, in: 2017 International Conference on Information Networking, ICOIN, IEEE, 2017, pp. 712–717.
[49] X. Wang, S. Chen, J. Su, App-net: A hybrid neural network for encrypted mobile traffic classification, in: IEEE INFOCOM 2020 - IEEE Conference on Computer Communications Workshops, INFOCOM WKSHPS, 2020, pp. 424–429.



Junjie Luo received the B.E. degree from the School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan, China, in 2021. He is currently pursuing the Ph.D. degree at the Institute of Computing Technology, Chinese Academy of Sciences, and Zhongguancun Laboratory. His research interests include edge intelligence, encrypted traffic analysis and network security.



Min Liu (Senior Member, IEEE) received her Ph.D. degree in computer science from the Graduate University of the Chinese Academy of Sciences, China. Before that, she received her B.S. and M.S. degrees in computer science from Xi'an Jiaotong University, China. She is currently a professor at the Institute of Computing Technology, Chinese Academy of Sciences, and also holds a position at the Zhongguancun Laboratory. Her current research interests include mobile computing and edge intelligence.



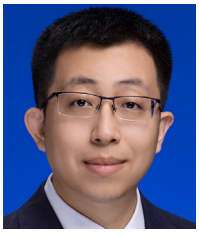
Sheng Sun received her B.S. and Ph.D. degrees in computer science from Beihang University, China, and the University of Chinese Academy of Sciences, China, respectively. She is currently an associate professor at the Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China. Her current research interests include federated learning, mobile computing and edge intelligence.



Yali Chen received the B.S. degree in communication engineering from the Taiyuan University of Science and Technology, China, in 2016 and the Ph.D. degree in communication and information systems from Beijing Jiaotong University, China, in 2022. She is an assistant professor with the Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China. Her current research interests include uncrewed aerial vehicles, over-the-air computation and edge intelligence.



Qixun Tang received the B.E. degree from Tsinghua University, Beijing, China, in 2023. He is currently pursuing the Ph.D. degree at the Institute of Computing Technology, Chinese Academy of Sciences, and Zhongguancun Laboratory. His research interests include network and system security.



Yuwei Wang received the Ph.D. degree from the University of Chinese Academy of Sciences, Beijing, China, in 2020. He is currently a senior engineer at the Networking Technology Research Centre, Institute of Computing Technology, Chinese Academy of Sciences. His current research interests include future network, virtualization technology and cloud computing.